

Step 3 — Docker + Rate Limiting — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

.dockerignore

```
.env
.venv
__pycache__
*.db
.git
```

api/rate_limit.py

```
from fastapi import HTTPException

_request_counts = {}

def check_rate_limit(client_id, limit=10):
    _request_counts[client_id] = _request_counts.get(client_id, 0) + 1
    if _request_counts[client_id] > limit:
        raise HTTPException(status_code=429, detail="Too many requests")
```

```
docker build -t contentforge-api .
docker run --env-file .env -p 8000:8000 contentforge-api
```

```
docker history contentforge-api
```

This works — the app builds and runs from a container, secrets pass in only at `docker run` time, and a basic request cap exists. The rate-limit counts never reset (no timer), so after `limit` total requests ever, that client is locked out permanently until the process restarts — a real bug, fixed in Intermediate.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — layer-cached Dockerfile, per-client fixed-window limiter

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

api/rate_limit.py

```
import time
from collections import defaultdict
from fastapi import HTTPException

WINDOW_SECONDS = 60
MAX_REQUESTS_PER_WINDOW = 10
_window_start: dict[str, float] = {}
_request_counts: dict[str, int] = defaultdict(int)

def check_rate_limit(client_id: str) -> None:
    now = time.monotonic()
    start = _window_start.get(client_id)

    if start is None or now - start > WINDOW_SECONDS:
        _window_start[client_id] = now
        _request_counts[client_id] = 0

    _request_counts[client_id] += 1
    if _request_counts[client_id] > MAX_REQUESTS_PER_WINDOW:
        raise HTTPException(status_code=429, detail="Too many requests")
```

api/routes.py (excerpt)

```

from fastapi import Request
from api.rate_limit import check_rate_limit

@router.post("/run-task", response_model=TaskResponse)
def run_task(request: TaskRequest, http_request: Request) -> TaskResponse:
    check_rate_limit(http_request.client.host)
    ...

```

Difference from Basic: the counter now resets on a rolling 60-second window per client instead of counting forever — `COPY requirements.txt . / RUN pip install ...` happen *before* `COPY .`, so Docker's build cache reuses the installed-dependencies layer on every rebuild where only your application code changed, not your `requirements.txt`, which makes iterating on the app during development much faster. `check_rate_limit` keys off the caller's IP (`http_request.client.host`), so different clients don't share one global limit. Still has the fixed-window boundary problem: a client can send `MAX_REQUESTS_PER_WINDOW` requests in the last second of one window, then `MAX_REQUESTS_PER_WINDOW` more in the first second of the next — double the intended rate, right at the boundary.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — sliding-window limiter with `Retry-After`

```
import time
from collections import defaultdict, deque
from fastapi import HTTPException

WINDOW_SECONDS = 60
MAX_REQUESTS_PER_WINDOW = 10
_request_times: dict[str, deque[float]] = defaultdict(deque)

def check_rate_limit(client_id: str) -> None:
    now = time.monotonic()
    timestamps = _request_times[client_id]

    while timestamps and now - timestamps[0] > WINDOW_SECONDS:
        timestamps.popleft()

    if len(timestamps) >= MAX_REQUESTS_PER_WINDOW:
        retry_after = int(WINDOW_SECONDS - (now - timestamps[0]))
        raise HTTPException(
            status_code=429,
            detail="Too many requests",
            headers={"Retry-After": str(max(retry_after, 1))},
        )

    timestamps.append(now)
```

Expected behavior: sending `MAX_REQUESTS_PER_WINDOW` requests back to back, then one more, gets a `429` with a `Retry-After` header telling you almost exactly how many seconds until the oldest request in the window ages out — and, unlike a fixed window, there's no boundary moment where a client can legitimately send double the intended rate.

Approach 2 — same idea, as `slowapi` (a maintained library instead of hand-rolled state)

```

from slowapi import Limiter
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

limiter = Limiter(key_func=get_remote_address)
app = FastAPI(title="ContentForge API")
app.state.limiter = limiter

@app.exception_handler(RateLimitExceeded)
def rate_limit_handler(request: Request, exc: RateLimitExceeded):
    return JSONResponse(status_code=429, content={"detail": "Too many requests"})

@app.post("/run-task")
@limiter.limit("10/minute")
def run_task(request: Request, task_request: TaskRequest):
    ...

```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's fixed window allows a real burst-at-the-boundary loophole. Both Advanced approaches close it, tracking actual request times in a rolling window rather than a counter tied to a fixed reset point. Approach 1 is the hand-rolled version — a `deque` per client, no new dependency, full visibility into exactly how it works, which matters for a project whose whole point is understanding these mechanics rather than trusting a black box. Approach 2 swaps in `slowapi`, a small, maintained library built specifically for this — sliding-window logic, `Retry-After` handling, and per-route limit strings (`"10/minute"`) all included, in exchange for one more dependency.

Which one should you actually write? Approach 1 for this project. The whole point of hand-building the sliding-window check is the same reason the `.env` parsing exercise had you write a parser by hand before reaching for `python-dotenv` — once you've built it yourself, `slowapi` (or any real rate-limiting middleware) stops being a black box and starts being "oh, that's the deque-of-timestamps trick, packaged." In a real production system with more than one route needing limits and more nuanced rules (per-user tiers, different limits per endpoint), reach for `slowapi` or similar — same trade-off Doc01's `.env` exercise reached for `python-dotenv` at the end: understand it by hand once, then use the well-tested library going forward.